

FathomNet 2026 — Master Plan v4

Today: Friday, April 17, 2026 **Kaggle deadline:** May 7, 2026 (20 days) **Budget:** up to \$300 (target ~\$230, leave ~\$70 headroom)

What this document is: Strategic guide — the *why*, *when*, and *how-to-decide* for every technique. Code is intentionally sparse (signatures + key math only). Implementation details belong in Cursor with this PDF as context.

Changelog from v3: - Clarified the evaluation metric is **COCO mAP@[.50:.95]**, not mAP@0.5. This changes priority rankings substantially — localization quality matters as much as classification. - Reranked differentiators by expected gain *under the actual metric* (tight boxes matter more). - Explicit note: no bulk image archive. Phase 0 must kick off `download.py` immediately. - Explicit note on rules: “Other outside data” is allowed but extra labeled marine images are not. - Per-class NMS IoU tuning (previously backup-only) promoted to higher priority.

Do This First (April 17–18)

1. **Register for CLEF by April 23** via the official lab registration form at `clef-labs-registration.dipintra.it/registrationForm.php`. Note: this is the CLEF *lab* registration (free). Having a Kaggle account alone is not enough — you need to appear on the lab’s registered-participants list for the official ranking.
2. **Join the Kaggle competition** at `kaggle.com/competitions/fathomnet-2026` and accept the rules.
3. **Accounts:** RunPod (add \$100 credit to start), W&B (free), Kaggle API token downloaded.
4. **Tooling:** Cursor with Remote-SSH extension installed, SSH key generated and added to RunPod.
5. **Kick off `download.py` immediately** — no bulk archive exists. The script downloads images one-by-one from the FathomNet server. Runs several hours. Start it *before* EDA; EDA only needs the JSON files you already have.
6. **Start Phase 0 EDA on the JSON files** while downloads run in parallel — no GPU needed.

Miss CLEF registration = no official ranking regardless of Kaggle score.

Metric Reality — What You’re Actually Optimizing

The competition is evaluated on **COCO mAP@[.50:.95]** — the mean of AP computed at 10 IoU thresholds (0.50, 0.55, 0.60, ..., 0.95) averaged across the 32 classes. This is significantly stricter than mAP@0.5 (the metric a lot of object detection tutorials use by default).

Why this changes priorities

Under mAP@0.5, a detection with 55% IoU and one with 85% IoU both count equally as “correct”. Under mAP@[.50:.95], the tighter detection is worth ~3–4× more in final score. Two implications:

1. **Tight, well-localized boxes matter as much as correct class labels.** A classifier fix that bumps your precision by 2% but loosens your boxes by 5% can *hurt* your score.
2. **Anything that improves box quality gets a priority bump.** Specifically:
 - **SAHI tiled inference** (Phase 7) — critical. Small objects at low IoU are typical failure modes; SAHI addresses this.
 - **Multi-scale training** (Phase 4) — more valuable than under mAP@0.5 because the model learns to localize at many scales.
 - **Score calibration** (Phase 7) — temperature scaling and isotonic matter more because mAP integrates over recall, so the *ordering* of predictions across the full confidence range affects every IoU threshold simultaneously.
 - **Per-class NMS IoU tuning** (now in main plan, see Phase 7) — promoted from backup. Different classes need different NMS aggressiveness for optimal localization.
 - **Weighted Box Fusion (WBF) in Phase 7 ensemble** — specifically prefer WBF over traditional NMS-based ensembling, because WBF *averages* coordinates from agreeing models. That averaging produces tighter, more accurate boxes. Under a pure mAP@0.5 metric the benefit would be marginal, but at IoU=0.9 the difference between “averaged box” and “kept-one box” is the difference between true positive and false positive.

What stays the same

- Class imbalance handling (EFL, RFS, copy-paste) — still critical for per-class AP
- PU learning (Kiryō, Soft Teacher) — still the core research problem
- SSL pretraining — helps both classification AND localization

What should be reprioritized downward

- Hierarchical taxonomy loss — it helps classification confusion but does nothing for box quality. Keep it but weight it below SAHI and calibration when allocating Phase 7 time.
-

Budget Allocation — up to \$300 (target ~\$230)

Phase	Where	GPU	Hours	Cost
0 — EDA + setup + download	Local (Cursor)	None	0	\$0
1 — SSL pretraining	RunPod A100 80GB	A100 80GB	~18	~\$45
2 — Baselines	Kaggle free	T4	~4	\$0
3 — PU learning + class imbalance	RunPod RTX 4090	RTX 4090	~15	~\$12
4 — Architecture + augmentation	RunPod RTX 4090	RTX 4090	~24	~\$18
5 — Hierarchical + pseudo R2	Mix (T4 + 4090)	T4 + 4090	~8	~\$5
6 — 5-fold + RT-DETR full	RunPod A100 40GB	A100 40GB	~40	~\$80
7 — SAHI + TTA + WBF ensemble + calibration + per-class NMS	RunPod RTX 4090	RTX 4090	~18	~\$14
Buffer	—	—	—	~\$60
Target total			~127 hours	~\$234
Headroom available				~\$66

Phase 7 expanded by ~3 hours to accommodate per-class NMS tuning — that’s the prioritization shift from the mAP@[.50:.95] analysis.

Timeline — 20 Days

Days	Dates	Phase	Focus
1-2	Apr 17-18	0	EDA + CLEF registration + download kickoff + environment setup
3-5	Apr 19-21	1	SSL pretraining
6-7	Apr 22-23	2	Baselines + first submission
8-10	Apr 24-26	3	PU + class imbalance experiments
11-13	Apr 27-29	4	Architecture + augmentation ablations
14	Apr 30	5	Hierarchical loss + pseudo round 2
15-18	May 1-4	6	Full training (5-fold + RT-DETR)
19	May 5	7	SAHI + WBF ensemble + calibration + per-class NMS
20	May 6	Buffer	Final submission, last fixes
21	May 7	DEADLINE	Submit by 11:59 PM UTC

Golden rule: One variable changed per experiment. Attribution is impossible otherwise.

Data Rules — What’s Allowed

The competition rules draw a critical distinction:

- **Extra *labeled* marine images:** NOT ALLOWED. You cannot supplement the 6,463 training images with additional annotated marine images.
- **Extra *unlabeled* data and pre-trained resources:** ALLOWED. This includes:

- Pre-trained backbones (ImageNet, BioClip2, DINOv2, SimCLR on the full FathomNet DB)
- Taxonomic trees and reference data for hierarchical loss
- Self-supervised learning on the full unlabeled FathomNet DB
- Metadata, captions, auxiliary information

This is permissive for our plan. Every Phase 1 SSL option (SimCLR, DINOv2, BioClip2) is allowed. The Phase 5 hierarchical loss is allowed. The Phase 3 Soft Teacher on the FathomNet DB is allowed.

One thing it rules out: you can't pull additional labeled marine detection datasets (e.g., VIAME benchmarks) to augment training. Don't do it even accidentally.

One structural fact

All 6,463 training images have at least one annotation. **There are no background-only images in training.** This confirms the plan's PU framing — every unlabeled region is *inside* an annotated image, meaning it's exactly the "maybe background, maybe unlabeled positive" ambiguity the Kiryo PU loss and Soft Teacher are designed to handle.

Phase 0's "single-category-label rate" metric is the right severity gauge for this.

The Three Core Challenges

FathomNet 2026 combines three problems at once, and a good solution attacks all three. Understanding which differentiator fights which problem is the key to prioritization.

Problem 1 — Positive-Unlabeled (PU) labels

Marine biologists only annotate what they personally specialize in. A jellyfish expert carefully boxes jellies while ignoring every other organism in frame. So for any unlabeled region, you can't tell whether it's background or a hidden positive. Standard detection losses treat unlabeled regions as confident negatives, which is wrong here.

Signal in the data: Count images with only one category labeled. If more than 40% of annotated images have a single category, PU is severe. FathomNet 2026 will almost certainly be in this regime.

Problem 2 — Long-tail class imbalance

32 categories, but frequencies span orders of magnitude. Common organisms (bony fish, sponges) dominate; rare ones (octopus, certain siphonophores) have tens of examples. Standard cross-entropy trains the model to favor head classes. Rare classes get drowned out by negative gradients from frequent classes.

Signal in the data: Compute `max_class_count / min_class_count` . If greater than 10, imbalance is significant. If greater than 100, it's extreme.

Problem 3 — Small-object / scale shift / localization quality

The 2025 post-mortem explicitly called out that test objects were on average smaller than training objects. Plus many marine organisms are naturally small relative to the frame. Standard object detectors struggle with small objects because the feature map at detection resolution has too few pixels covering them. And under mAP@[.50:.95], loose localization on small objects compounds into major score loss.

Signal in the data: Count objects with max-side length under 64 pixels, or relative area under 1%. If more than 25%, small objects are a major concern.

How differentiators map to problems

Differentiator	PU	Imbalance	Small-obj / localization
Kiryo PU loss	[YES]		
Soft Teacher	[YES]		
Hierarchical loss		[YES]	
Equalized Focal Loss (EFL)		[YES]	
Repeat Factor Sampling (RFS)		[YES]	
Class-aware copy-paste		[YES]	
Underwater preprocessing			[YES]
Multi-scale training			[YES]
SAHI tiled inference			[YES]
Per-class NMS IoU tuning			[YES]
Context crops		[YES]	

SSL pretraining	[YES]	[YES]	[YES]
TTA + WBF ensemble + calibration	[YES]	[YES]	[YES]

SSL pretraining and the inference-time stack help with all three — those are universally valuable. Everything else is targeted: prioritize based on which problem your EDA shows to be most severe.

Phase 0 — EDA and Setup

Days 1-2 • Local Cursor • No GPU • \$0

Two parallel workstreams

Stream A — runs overnight unattended: 1. Kick off `download.py` against `dataset_train.json`. Let it run. Several hours. 2. Once train images complete, start `dataset_test.json` download. Another ~1 hour. 3. Verify downloaded image count matches expected (~6,463 train, ~1,425 test).

Stream B — you work on this during and after the download: 1. EDA on the JSON files (no images needed) 2. Compute all decision-gating metrics 3. Build train/val splits, RFS weights, taxonomy tree 4. Write `phase0_decisions.md` — one-page summary of which differentiators are justified

EDA only needs annotations. Don't wait for image downloads to finish to start computing metrics.

What you compute from JSON alone

1. **Class frequency distribution** — count of instances per category
2. **PU severity** — percentage of images labeled with only a single category
3. **Object size distribution** — absolute (pixels) and relative (% of frame)
4. **Image color statistics** — per-channel means, used to decide on underwater preprocessing. *This needs actual images, so defer until a subset has downloaded; ~100 sample images are enough.*

Decision table derived from EDA

Metric	Threshold	Action
Images with single category labeled	> 50%	Prioritize Soft Teacher, Kiryo PU (Phase 3)
Images with single category labeled	30-50%	Standard PU handling sufficient
Images with single category labeled	< 30%	PU may be mild — focus on imbalance
Class imbalance ratio (max/min)	> 50	EFL + RFS + copy-paste all essential
Class imbalance ratio (max/min)	10-50	EFL + RFS recommended
Class imbalance ratio (max/min)	< 10	Class weighting alone may suffice
Objects with max-side < 64 px	> 25%	imgsz=1280 baseline, SAHI critical
Objects with max-side < 64 px	10-25%	imgsz=1024 OK, SAHI helpful
Objects with max-side < 64 px	< 10%	Default imgsz=640, SAHI optional
Red channel mean < 0.7 × B or G mean	yes	Underwater preprocessing (gray-world) helps
Std dev of channel V (HSV) > 50	yes	CLAHE helps handle uneven lighting

Deliverables from Phase 0

- `data/class_counts.json` — per-class instance counts
- `data/rfs_repeat_factors.pkl` — RFS weights per image
- `data/cv_folds.pkl` — 5-fold stratified split indices
- `configs/fathomnet.yaml` — YOLO dataset config
- `configs/taxonomy.py` — marine taxonomy tree
- `notes/phase0_decisions.md` — one-page summary of thresholds hit and which differentiators Phase 0 data justifies
- **Images fully downloaded** and a successful image-loading test (open one random image from each split with OpenCV, confirm it decodes)

Project structure

```
fathomnet-2026/
+--- data/
|   +--- raw/                # download.py output lives here
|   +--- train_images/
|   +--- test_images/
|   +--- class_counts.json
|   +--- rfs_repeat_factors.pkl
|   +--- cv_folds.pkl
+--- src/
|   +--- efl_loss.py
|   +--- pu_loss.py
|   +--- hierarchical_loss.py
|   +--- soft_teacher.py
|   +--- rfs_sampler.py
|   +--- copy_paste.py
|   +--- underwater_preproc.py
|   +--- sahi_inference.py
|   +--- ensemble.py
|   +--- calibration.py
+--- configs/
+--- notebooks/
+--- notes/
+--- submissions/
+--- docs/
```

Phase 1 — Self-Supervised Pretraining

Days 3-5 • RunPod A100 80GB • ~18 hours • ~\$45

Goal

Pretrain a backbone on the 400k+ unlabeled FathomNet images so the detector in Phase 2+ starts from features already tuned to underwater imagery, rather than ImageNet features that learned about dogs and cars.

Design rationale

Competition training set is only 6,463 images. Most competitors fine-tune from ImageNet weights, so their backbone hasn't seen underwater imagery. Self-supervised pretraining on the full FathomNet database — which needs no labels — gives you a backbone that already understands marine visual patterns (color shifts with depth, backscatter, marine organism textures) before you ever show it a label.

Architecture decision: three paths

Path	Effort	Cost	Expected benefit
BioClip2 backbone	0 hrs	\$0	+0.5–1.5 mAP over ImageNet
SimCLR on FathomNet DB	~10 hrs	~\$25	+1–2 mAP over ImageNet
SimCLR + DINOv2 fine-tune	~18 hrs	~\$45	+1.5–3 mAP over ImageNet

BioClip2 is already pretrained on FathomNet data. It's the free shortcut. Full SSL pretraining on the raw DB is slightly better but requires GPU time. DINOv2 fine-tuning pushes it further with modern representation learning.

Include / skip decision

```
IF time_budget_remaining_after_setup >= 3 days AND gpu_budget >= $40:
    DO full SSL pretraining (SimCLR or SimCLR + DINOv2)
ELIF time_budget_remaining >= 1 day:
    DO SimCLR only (skip DINOv2)
ELSE:
    SKIP — use BioClip2 backbone from Hugging Face
```

Key design choices within SimCLR

- **Augmentation strength:** Stronger than standard because underwater images need more color variation to learn invariance to depth-dependent color shifts
- **Batch size:** 512+ on A100 80GB. Contrastive learning's quality scales with number of negative pairs per batch
- **Temperature:** 0.07 (standard SimCLR value)
- **Epochs:** 100 is typical; 50 is acceptable if time-constrained

Light code reference

```
# Core contrastive loss pattern (NT-Xent)
# z1, z2 are normalized projections of two augmented views
loss = -log(exp(sim(z1, z2) / T) / sum_k(exp(sim(z1, z_k) / T)))
# T = temperature; sum over all negatives in batch
```

Success signal

After Phase 2, compare val mAP@[.50:.95] of three runs: (1) ImageNet backbone, (2) BioClip2 backbone, (3) SimCLR/DINOv2 backbone. If path 3 > path 2 > path 1, Phase 1 worked. If path 3 ≤ path 2, SSL didn't add value over the free shortcut — use BioClip2 going forward and don't re-run.

Failure mode

SSL loss doesn't decrease smoothly or plateaus early — usually means augmentation is too weak (views too similar, trivial to match) or batch size too small. Increase color jitter strength or move to a bigger batch.

Phase 2 — Baselines

Days 6-7 · Kaggle free T4 · ~4 GPU hours · \$0

Goal

Establish your reference point on the leaderboard. Every subsequent experiment is measured against Phase 2 mAP@[.50:.95].

Architecture: YOLOv11m with default settings

Medium size (m) is the sweet spot — large enough to capture fine features, small enough to iterate quickly. YOLOv11 reads COCO natively via Ultralytics, so integration is minimal.

Experiments

1. **Nano sanity check** (YOLOv11n, 10 epochs, ~30 min) — just verifies pipeline runs end-to-end
2. **Medium baseline** (YOLOv11m, 50 epochs, imgsz=640, ~2h) — true reference point
3. **Backbone bake-off** (~1.5h) — same training, three different starting weights: ImageNet, BioClip2, and your SSL backbone from Phase 1

Key decision

Take the baseline and the winning backbone from the bake-off. That becomes your foundation for every later experiment.

```
IF SSL_backbone_val_mAP > BioClip2_backbone_val_mAP + 0.5:
    USE SSL backbone going forward
ELIF BioClip2_backbone_val_mAP > ImageNet_val_mAP + 0.3:
    USE BioClip2 going forward (don't waste further SSL compute)
ELSE:
    USE ImageNet (none of the pretrained backbones help for some reason – worth investigating, could be a loading bug)
```

Submit to Kaggle

Submit your best Phase 2 run. Record leaderboard mAP@[.50:.95]. This is your starting line.

Failure mode

Val mAP much lower than expected for baseline YOLO (< 0.15). Usually a data conversion bug — check that COCO -> YOLO coordinate conversion is correct, that bounding boxes are normalized to [0,1], and that class indices are 0-indexed.

Phase 3 — PU Learning + Class Imbalance

Days 8-10 · RunPod RTX 4090 · ~15 GPU hours · ~\$12

Goal

Attack the two biggest problems simultaneously: incomplete labels (PU) and long-tail imbalance.

Loss architecture overview

YOLO's total loss has three components: box regression, objectness, classification. Your Phase 3 end-state replaces the default cross-entropy with specialized losses on each applicable component:

- **Box regression:** unchanged (CioU / DFL loss)
- **Objectness:** replace standard BCE with **Kiryo PU loss** — addresses the “is this really background?” question
- **Classification:** replace standard BCE with **Equalized Focal Loss (EFL)** — addresses long-tail imbalance

On the data side, **Repeat Factor Sampling (RFS)** oversamples images containing rare classes before they even reach the loss. So you're attacking imbalance both at the loss level (EFL) and at the sampling level (RFS).

Exp 3.1 — Estimate PU prior π (no GPU, CPU only)

The Kiryo loss needs an estimate of π — the fraction of unlabeled regions that actually contain hidden positives.

```
#  $\pi$  estimation (conceptual)
pi ≈ (annotated_image_rate) × (mean_annotated_area_fraction)
# Typical range: 0.1 - 0.4
```

Exp 3.2 — Kiryo PU objectness loss (~2h)

Problem: Standard BCE treats unlabeled regions as confident negatives, but a fraction π of them contain hidden positives. Training with this wrong assumption suppresses detections that should be positive.

Design principle: Kiryo et al. 2017 showed you can get an unbiased estimator of the true negative loss from only positives and unlabeled samples:

```
true_negative_loss ≈ unlabeled_loss -  $\pi$  × positive_loss
```

Plus a non-negativity correction to prevent the corrected loss from going below zero (which causes overfitting).

Decision criteria:

```
IF phase_0_single_category_rate > 40%:
    INCLUDE Kiryo PU loss — PU problem is severe enough to warrant proper handling
ELIF phase_0_single_category_rate 20-40%:
    INCLUDE as a moderate-benefit play
ELSE:
    SKIP or USE simpler approximation (reduce `cls` loss weight in Ultralytics config)
```

Success signal: Val mAP@[.50:.95] improves by 1+ point vs baseline, AND precision stays stable or improves (you're not just predicting more — you're predicting more accurately).

Failure mode: Val mAP drops. Usually π is over-estimated. Reduce π by half and re-run. Non-negative correction with `beta=0` is default; raising `beta` to 0.1-0.5 can also stabilize.

Exp 3.3 — Equalized Focal Loss (~2h)

Problem: Rare classes get drowned out. Standard focal loss uses one focusing factor γ for all classes, but rare classes actually need more focusing than frequent ones.

Design principle: EFL uses a per-class focusing factor that scales with how suppressed that class is:

```
 $\gamma_c = \gamma_{base} + s \times (1 - g_c)$ 
```

where `g_c` is the cumulative positive/negative gradient ratio for class `c` (tracked during training). Rare classes that get heavily suppressed have `g_c -> 0`, so `$\gamma_c$` grows — more focusing pressure on their hard examples.

Architecture integration: Replaces the classification head's loss function. `$\gamma_{base}=2$` (same as default focal loss), `scale_factor=8` (EFL paper default).

Decision criteria:

```
IF class_imbalance_ratio > 10:
    INCLUDE EFL — class imbalance significant enough to benefit
IF class_imbalance_ratio > 50:
    ESSENTIAL — standard cross-entropy will fail on tail classes
ELIF class_imbalance_ratio 5-10:
    OPTIONAL — plain focal loss may suffice
ELSE (ratio < 5):
    SKIP — dataset is balanced enough
```

Success signal: After training, compute per-class AP. Rare classes (the bottom 10 by instance count) should improve by 2-5 AP points vs baseline, while frequent-class AP stays within 1 point of baseline.

Failure mode: If frequent-class AP drops > 2 points, `scale_factor=8` is too aggressive. Reduce to 4. If rare-class AP doesn't improve, the gradient ratio tracking may be too noisy for small batches — increase the exponential moving average decay.

Exp 3.4 — Repeat Factor Sampling (~1h)

Problem: Even with EFL, if images containing rare classes are rarely sampled during training, the model never sees enough examples to learn them.

Design principle: RFS assigns each image a repeat factor based on the rarest class it contains. Rare-class images appear in training batches more often.

```
# Repeat factor per class
rf_c = max(1.0, sqrt(t / f_c))
# f_c = class frequency, t = threshold (typically 0.001)
# Per-image repeat factor = max over classes present
rf_img = max(rf_c for c in classes_in_image)
```

Architecture integration: Replace Ultralytics’ default random sampler with a `WeightedRandomSampler` using `rf_img` as weights.

Decision criteria:

```
IF >5 classes have <100 training instances:
    INCLUDE RFS — rare classes need oversampling
ELIF >2 classes have <50 instances:
    INCLUDE with higher threshold t=0.005 (more aggressive oversampling)
ELSE:
    SKIP — natural sampling is fine
```

Pairs with EFL: If EFL helped, RFS almost always helps further. Run them together as default.

Success signal: Rare-class AP improves additively on top of EFL gains. Training loss curves for rare classes should now show much smoother descent.

Failure mode: If overall mAP regresses, `t` is too aggressive. Reduce from 0.001 to 0.0001.

Exp 3.5 — Soft Teacher (~4h)

Problem: Pseudo-labeling (the semi-supervised approach from the original plan) uses a static teacher and hard thresholds. Modern approach is continuous adaptation.

Design principle: A “teacher” model, updated as an exponential moving average (EMA) of the student, generates soft pseudo-labels on-the-fly every batch. The student learns from both labeled data AND confidence-weighted teacher predictions.

```
# EMA update, every step
teacher =  $\alpha$  × teacher + (1 -  $\alpha$ ) × student    #  $\alpha$  = 0.9996 typical
# Student loss combines:
total_loss = sup_loss_on_labeled +  $\lambda$  × unsup_loss_weighted_by_teacher_confidence
```

Architecture integration: Two copies of the model. Student gets gradients, teacher is frozen per-batch but EMA-updated after. Unlabeled batch goes through teacher (for pseudo-labels) and student (for learning).

Decision criteria:

```
IF phase_0_single_category_rate > 30% AND you have unlabeled FathomNet imagery:
    INCLUDE Soft Teacher
IF plain pseudo-labeling round 1 improved val mAP by > 1 point:
    UPGRADE to Soft Teacher for iterative gain
ELIF pseudo-labels hurt performance:
    SKIP Soft Teacher — teacher will just amplify the problem
```

Note on unlabeled data: There are no background-only training images (all 6,463 have ≥ 1 annotation). But the “unlabeled data” for Soft Teacher is *regions within annotated images* that have no box — which is exactly what Kiryo PU handles on the objectness side. Soft Teacher operates on the full images from the FathomNet DB beyond the competition training set (allowed under “other outside data”). You can also treat high-confidence unmatched predictions within competition images as pseudo-positives.

Implementation reality check: Ultralytics doesn’t have native Soft Teacher. You have two practical paths: 1. **Full Soft Teacher** — fork Ultralytics, add teacher model + EMA update + dual-path loss. ~1 day of integration work. 2. **Poor-man’s Soft Teacher** — run 2 rounds of pseudo-labeling where the second round uses EMA-averaged weights between rounds. Gets ~60% of the benefit for ~20% of the effort.

Given your 20-day timeline, start with poor-man’s version. Upgrade only if it gives positive signal and time permits.

Success signal: Val mAP@[.50:.95] improves 2–4 points over Kiryo-only PU baseline.

Failure mode: Teacher predicts noise, student learns noise, mAP plummets. Confidence threshold for teacher pseudo-labels is too low. Raise from 0.5 to 0.7.

Exp 3.6 — Combined: EFL + RFS + Kiryo PU + Soft Teacher (~3h)

Run all four together. This is the Phase 3 graduation experiment — your best-of-Phase-3 config feeds into Phase 4.

Phase 3 exit decision

```
IF combined_val_mAP > baseline + 3:
    PROCEED to Phase 4 with full Phase 3 stack
ELIF combined_val_mAP > baseline + 1:
    ANALYZE per-differentiator ablation results – keep the ones that moved mAP positively, drop the ones that didn't
ELSE:
    DEBUGGING – something is wrong (loss masking bug, data leakage, config error). Don't proceed until Phase 3 beats baseline.
```

Phase 4 — Architecture and Augmentation

Days 11-13 · RunPod RTX 4090 · ~24 GPU hours · ~\$18

Goal

Identify the best combination of input resolution, data augmentation, and detector architecture. Each experiment isolates one variable.

Exp 4.1 — Image resolution (1280 vs 640) (~2h)

Design principle: Larger input = more pixels per object = better small-object detection AND tighter localization. Trades GPU memory for accuracy. Under mAP@[.50:.95], the localization benefit is magnified.

Decision criteria:

```
IF phase_0_small_object_rate > 25%:
    USE imgsz=1280 as default (small objects are common)
IF phase_0_small_object_rate 10-25%:
    IF GPU memory allows: imgsz=1024
    ELSE: imgsz=640 + rely on SAHI at inference
IF imgsz=1280 val mAP > imgsz=640 val mAP + 1:
    LOCK imgsz=1280 for all subsequent training
```

Exp 4.2 — Underwater preprocessing (~2h)

Problem: Underwater images have depth-dependent color cast (red absorbed first -> blue/green shift) and uneven ROV lighting. Standard ImageNet-tuned backbones weren't trained on this distribution.

Design principle: Apply two classical preprocessing steps before the model sees the image: - **Gray-world color correction** — normalize per-channel means to remove color cast - **CLAHE on L channel (LAB space)** — handle uneven lighting via local contrast enhancement

Applied at both training and inference time.

Light code reference:

```
img = gray_world(img)          # remove color cast
img = clahe_on_L_channel(img)  # local contrast enhancement
```

Decision criteria:

```
IF phase_0_red_channel_mean < 0.7 * (green + blue) / 2:
    INCLUDE gray-world (depth cast is significant)
IF phase_0_image_V_stddev > 50:
    INCLUDE CLAHE (lighting varies across dataset)
IF val_mAP_with_preproc > val_mAP_without_preproc + 0.5:
    LOCK preprocessing in the pipeline for all subsequent experiments
ELSE:
    DROP — preprocessing wasn't helping, model had learned it implicitly
```

Exp 4.3 — Underwater augmentation (~2h)

Design principle: Augmentation teaches the model what variation is irrelevant. Underwater-specific augmentation: - Hue jitter (color cast varies with depth) - Brightness and saturation (ROV lights vary) - Random rotation (organisms have no canonical orientation underwater) - Vertical flips (up/down isn't as fixed as in land photos)

Built into Ultralytics as `hsv_h`, `hsv_s`, `hsv_v`, `degrees`, `flipud`, `fliplr` hyperparameters. No custom code needed — just aggressive settings.

Decision: Always include. Ultralytics defaults are tuned for COCO (terrestrial), so underwater calibrated settings help.

Exp 4.4 — Class-aware copy-paste (~3h)

Problem: Even with RFS, rare classes may only have 30–50 total instances. Copy-paste synthesizes more training instances by extracting crops from rare-class images and pasting them into other training images.

Design principle: Standard copy-paste picks random instances. Class-aware copy-paste biases the selection toward rare classes.

Architecture integration: Wraps the dataset's `__getitem__`. With probability $p=0.5$, picks a rare-class instance from a pre-built bank and pastes it into the current training image at a non-overlapping location.

Decision criteria:

```
IF >3 classes have <50 training instances:
    INCLUDE class-aware copy-paste
IF copy-paste augmented images look unnatural (pasted crop clearly out of context):
    REDUCE prob to 0.2
IF rare_class_AP improves > 1 point without frequent_class_AP drop:
    KEEP
ELSE:
    REDUCE prob or DROP
```

Exp 4.5 — Multi-scale training (~3h)

Problem: The 2025 distribution shift — test objects smaller than training objects. If you train at fixed `imgsz=1280`, the model sees a narrower scale distribution than it will face at test time.

Design principle: Randomly vary input resolution per batch. Ultralytics has `multi_scale=True` built-in (varies `imgsz` $\pm 50\%$). Plus bias `scale=(0.3, 1.0)` instead of default `(0.5, 1.5)` to emphasize *smaller* test objects.

Under mAP@[.50:.95] this is especially valuable — the model that learned to localize well at multiple scales produces tighter boxes at test scale, which pays off at higher IoU thresholds.

Decision criteria:

```
IF phase_0_small_object_rate > 20%:
    INCLUDE multi-scale training with scale=0.5
IF test leaderboard mAP < val mAP by > 2 points:
    Scale shift is real — ENABLE multi-scale + reduce scale range toward smaller
ELSE:
    Multi-scale is a default-on — minimal downside
```

Exp 4.6 — Larger model YOLOv11l (~3h)

Decision criteria:

```
IF YOLOv11m val_mAP is plateauing (no gain after 30+ epochs):
    TRY YOLOv11l
IF YOLOv11l val_mAP > YOLOv11m val_mAP + 1:
    USE YOLOv11l for Phase 6 full training
ELSE:
    KEEP YOLOv11m — larger model's slower iteration outweighs marginal accuracy gain
```

Exp 4.7 — RT-DETR (~3h)

Design principle: Transformer-based detector. Different inductive biases from YOLO -> different failure modes -> better ensemble partner.

Decision criteria:

```
IF RT-DETR solo val_mAP within 2 points of YOLOv11m:
    INCLUDE in Phase 7 ensemble (architectural diversity helps)
IF RT-DETR > 5 points behind YOLOv11m:
    SKIP ensemble inclusion — it'll drag the average down
```

Exp 4.8 — Multi-scale context crops (~2h)

Design principle: 2025 winning insight. Feed the model both the full image AND padded crops around each detection. Context tells the classifier whether a round gelatinous thing is a jelly (midwater) or a different species (near seafloor).

Decision criteria:

```
IF confusion matrix shows high confusion between classes that differ by habitat:
    INCLUDE context crops
ELIF confusion is random across unrelated classes:
    SKIP — context doesn't disambiguate random errors
```

Context crops are more complex to integrate. Only include if simpler techniques have been exhausted and you have time.

Phase 4 exit deliverable

Pick the 1–2 best configs from Phase 4. These seed Phase 6 full training.

Phase 5 — Hierarchical Loss + Pseudo Round 2

Day 14 · Mix Kaggle T4 + RunPod · ~8 GPU hours · ~\$5

Goal

Exploit marine taxonomy structure — confusing sea star with brittle star (both echinoderms) should cost less than confusing sea star with jelly (different phyla). Standard cross-entropy treats all misclassifications equally.

Note on mAP@.50:.95: Hierarchical loss only affects classification. It does nothing for localization. So it's valuable but ranked *below* Phase 7's localization-improving techniques. Include it but don't sacrifice SAH or calibration time for it.

Design principle

Build a taxonomic distance matrix between the 32 classes (tree distance between leaves in the phylum -> class -> order -> family hierarchy). Add to cross-entropy a term proportional to the predicted distribution's expected distance from the true class:

```
loss = CE_loss + λ × sum_c(P(c | x) × dist(c, true_class))
```

$\lambda = 0.5$ is a reasonable starting weight.

Architecture integration

Replaces classification loss. Incompatible with EFL unless you carefully combine — use hierarchical loss as the classification component and keep EFL's per-class γ as a separate modulator on it.

Decision criteria

```
IF Phase 3 confusion matrix shows > 40% of errors are within-phylum:
    INCLUDE hierarchical loss — errors are structured, taxonomy helps
ELIF errors are evenly distributed across unrelated classes:
    SKIP — taxonomic structure isn't the bottleneck
IF hier_loss_val_mAP > Phase_3_best_val_mAP:
    KEEP in final pipeline
ELSE:
    DROP and use standard classification loss
```

Pseudo round 2

Using your best Phase 5 model, run pseudo-labeling a second time. Generate higher-quality labels from a better model -> retrain -> one more mAP gain.

Decision criteria:

```
IF pseudo round 1 gave > 1 mAP gain:
    RUN round 2
IF round 2 gives > 0.5 mAP gain:
    ACCEPT new labels for Phase 6 training data
IF round 2 regresses OR is < 0.3 gain:
    STOP at round 1 — diminishing returns
```

Phase 6 — Full Training on Best Configs

Days 15–18 · RunPod A100 40GB · ~40 GPU hours · ~\$80

Goal

Train final models to convergence with all differentiators locked in. These are what go into the Phase 7 ensemble.

Design: 5-fold stratified CV + 1 full-data RT-DETR

Train 5 YOLOv11m models on 5 different 80/20 stratified splits. Each fold sees different training data, making the models' errors decorrelated — good for ensembling. Plus 1 RT-DETR trained on all data for architectural diversity.

Total: 6 models.

Why 5-fold, not train once on all data?

1. **Decorrelated errors** — different training sets -> different blind spots -> averaging helps more
2. **Robust validation** — mean mAP across folds is a reliable estimate; single-split val can be lucky
3. **Fallback safety** — if one fold has a bug or crash, 4 others still produced models
4. **Free ensemble** — you'd need 5 training runs for the ensemble anyway; 5-fold just uses that compute for both validation AND ensemble construction

Training configuration

All 5 folds use the same config (the Phase 5 winner). Differences between folds come only from the training data split.

```
per-fold config:
model: YOLOv11m (pretrained from SSL/BioClip backbone)
imgsz: 1280 (if Phase 4 showed it helps)
epochs: 150
loss: EFL classification + Kiryo PU objectness + hierarchical (if Phase 5 won)
sampler: RFS
augmentation: underwater UW + class-aware copy-paste + multi-scale
teacher: Soft Teacher with EMA (if Phase 3 Soft Teacher won)
optimizer: cosine LR, close_mosaic=20
```

Exit criteria

```
IF fold_mAP_variance > 2 points across 5 folds:
    Investigate — some folds may have data issues (rare class absent from a fold's train set?)
IF mean_fold_mAP > Phase_5_best_val_mAP + 1:
    Phase 6 was worth it — proceed to Phase 7 with 6 models
ELSE:
    Phase 6 didn't add much — maybe 150 epochs was overkill, or the differentiator stack is saturated
```

Headroom usage decision

```
IF Phase 6 completes with > $40 buffer remaining AND time allows:
    The single best use is NOT more models — it's per-class threshold tuning
    (covered in Phase 7). No GPU needed. ~$0 cost. +0.5-1 mAP.
ELSE:
    Leave unspent. You have a good submission either way.
```

Phase 7 — SAHI + WBF Ensemble + Calibration + Per-Class NMS

Day 19 · RunPod RTX 4090 · ~18 GPU hours · ~\$14

This phase is where mAP@[.50:.95] is won or lost. Every technique here improves either box localization quality or ranking/calibration — both are magnified under the strict metric. Five techniques total, each adds mAP free of compute cost beyond inference.

Differentiator 1: SAHI tiled inference — HIGHEST PRIORITY

Problem: Small test objects + strict IoU thresholds. Your detector was trained at imgsz=1280, but an object occupying 40 pixels in a 1920-wide test image is only 27 pixels at input size 1280. The feature map at detection head resolution has maybe 2-3 cells covering it — not enough to produce a tight enough box to survive $\text{IoU} \geq 0.75$.

Design principle: Slice each test image into overlapping 640×640 tiles. Run detector on each tile. Merge predictions with NMS across tiles. Each tile is effectively a zoom-in; small objects become relatively large and get well-localized boxes.

Light code reference:

```
result = get_sliced_prediction(
    image, detection_model,
    slice_height=640, slice_width=640,
    overlap_height_ratio=0.2, overlap_width_ratio=0.2,
)
```

Decision criteria:

```
IF phase_0_small_object_rate > 20%:  
    INCLUDE SAHI — expected +2 to 5 mAP@.50:.95], higher under strict metric  
IF test LB mAP noticeably below val mAP:  
    INCLUDE SAHI — likely scale shift  
IF phase_0_small_object_rate < 5%:  
    SKIP — large objects don't benefit, SAHI just adds inference time
```

Differentiator 2: Test-time augmentation (TTA)

Design principle: Run inference with multiple augmentations (horizontal flip, vertical flip, 90° rotations). Merge with Weighted Box Fusion. Each augmentation sees the image differently; errors don't fully correlate.

Decision criteria:

```
ALWAYS INCLUDE unless inference time is strictly capped  
Expected gain: +0.5 to 1.5 mAP  
Cost: 5× inference time
```

Differentiator 3: 6-model WBF ensemble — NOT NMS

Design principle: Weighted Box Fusion merges predictions from multiple models by **averaging spatial coordinates** of overlapping boxes, weighted by confidence. This is meaningfully different from NMS-based ensembling, which keeps one box and drops the others.

Why WBF specifically matters under mAP@.50:.95]: Under mAP@0.5, a slightly loose box is still a true positive. Under mAP@.50:.95], the same loose box is a false positive at IoU=0.85 and a true positive at IoU=0.5. Averaging coordinates from 3–6 models that agree on location produces a tighter, more accurate box that counts as TP across more IoU thresholds. Under mAP@0.5 the benefit of WBF over NMS is ~+0.5 mAP. Under mAP@.50:.95] the benefit is typically +1–2 mAP.

Decision criteria per model:

```
IF solo_model_LB_mAP within 3 points of best solo:  
    INCLUDE in ensemble  
IF solo_model_LB_mAP 3–5 points behind best solo:  
    INCLUDE with reduced weight (0.5)  
IF > 5 points behind:  
    EXCLUDE — it'll drag the ensemble down
```

Weight tuning methodology: 1. Start equal weights 2. Grid search over RT-DETR weight $\in \{0.6, 0.8, 1.0, 1.2, 1.4\}$ on val data 3. Lock the weight that maximized val mAP@.50:.95]

Differentiator 4: Per-class NMS IoU tuning — PROMOTED FROM BACKUP

Problem: Default NMS uses one IoU threshold for all classes. But dense organisms (amphipod clusters, shrimp swarms) benefit from low NMS IoU (allow closely-overlapping distinct detections), while isolated large organisms (single octopus, large jelly) benefit from high NMS IoU (suppress duplicates of the same object). One global threshold can't be optimal for both.

Why this is promoted to main plan under mAP@.50:.95]: At higher IoU thresholds, suboptimal NMS decisions are more costly. If NMS merges two distinct nearby organisms because IoU=0.55 crossed the threshold, you lose both under strict IoU criteria (one is a missed detection, the other has wrong localization). Per-class tuning recovers these cases.

Design principle: Grid search NMS IoU threshold per class on validation data. Apply class-specific NMS at inference.

Decision criteria:

```
IF > 10% of test images have > 5 objects (from Phase 0 EDA):  
    INCLUDE per-class NMS — density varies significantly  
IF val confusion shows adjacent-class boxes being dropped:  
    INCLUDE — NMS is over-aggressive for dense classes  
ALWAYS CONSIDER — almost free, CPU-only tuning from validation predictions
```

Methodology: 1. On val predictions, sweep NMS IoU $\in \{0.3, 0.4, 0.5, 0.6, 0.7\}$ per class 2. For each class, compute AP at each threshold 3. Lock the threshold that maximized that class's AP 4. At inference: for each class, apply class-specific NMS with its tuned threshold

Expected gain: +0.5 to 1.5 mAP@.50:.95], more than typical backup items due to metric strictness.

Differentiator 5: Score calibration (isotonic + temperature)

Problem: Raw detector confidence isn't calibrated — score=0.9 doesn't mean 90% likely correct. mAP integrates over the entire precision-recall curve, so mis-calibrated scores hurt across every IoU threshold simultaneously. Under mAP@.50:.95], this compounds across 10 thresholds.

Design principle: Fit a monotonic map from raw score -> calibrated probability using val data. Apply at test time.

Try both and keep the winner: - **Isotonic regression** — flexible, captures non-monotonic miscalibration, higher capacity - **Temperature scaling** — single parameter, less prone to overfit, safer on small val sets

Decision criteria:

ALWAYS INCLUDE – fitting takes seconds, no downside
Can fit from the 5-fold out-of-fold predictions (free since you did 5-fold CV)
IF val set is small (< 5000 detections):
 PREFER temperature scaling
ELSE:
 TRY BOTH, keep winner on held-out fold

Expected gain under mAP@.50:.95]: +0.5 to 1 mAP (higher than under mAP@0.5 because calibration affects all IoU thresholds).

Experiments in Phase 7

- 1. Best single model + TTA only (~1h) — isolates TTA contribution
 - 2. Best single model + SAHI only (~1h) — isolates SAHI contribution
 - 3. Best single model + SAHI + TTA (~1h) — combined inference stack
 - 4. 5-fold YOLO ensemble + SAHI + TTA (~4h) — ensemble without RT-DETR
 - 5. 6-model ensemble + SAHI + TTA (~4h) — with RT-DETR
 - 6. 6-model ensemble + SAHI + TTA + **per-class NMS** (~3h) — tune per-class NMS IoU
 - 7. **Final: 6-model + SAHI + TTA + per-class NMS + calibration** (~4h) — the submission
- Submit each — each submission validates the pipeline and tells you which techniques actually moved the leaderboard.

Master Decision Tree Summary

All major include/skip decisions consolidated, reranked for mAP@.50:.95]:

Technique	Include IF	Skip IF
SSL pretraining (full)	Time ≥ 3 days AND budget ≥ \$40	Time-constrained -> use BioClip2
BioClip2 backbone	Always (it's free)	—
Kiryo PU loss	Single-cat rate > 40%	< 20% (PU mild)
EFL	Class imbalance > 10x	< 5x (balanced enough)
RFS	> 5 classes with < 100 instances	Classes balanced
Soft Teacher	PU severe AND unlabeled imagery available	Round 1 pseudo-labels hurt mAP
Hierarchical loss	> 40% of errors within-phylum	Errors random across classes
Pseudo round 2	Round 1 gained > 1 mAP	Round 1 regressed
Underwater preproc	Red channel much lower than G/B	Red channel near green/blue
Class-aware copy-paste	> 3 classes with < 50 instances	Pasted crops look unnatural
Multi-scale training	Small-object rate > 20%	— (default on, high value under strict mAP)
imgsz=1280	Small-object rate > 25%	< 10%
YOLOv11l	YOLOv11m plateaus	YOLOv11l not ≥1 mAP better
RT-DETR in ensemble	Solo mAP within 3 of YOLO	> 5 points behind
Context crops	Habitat-related confusion dominates	Simpler techniques not exhausted
5-fold CV	Always (unless extreme time crunch)	Last-day-only — single run + duplicate
SAHI inference	Small-object rate > 20%	< 5% (high value under strict mAP)
TTA	Always	Inference time capped
WBF ensemble (not NMS)	Always ensemble > 1 model	Only 1 model trained
Per-class NMS IoU	> 10% images with >5 objects	Objects always isolated
Calibration (isotonic or temp)	Always	—

Bolded rows are techniques whose priority increased under the strict metric.

Common Failure Modes and Fixes

--	--	--

Symptom	Likely cause	Fix
Val mAP plummets after adding PU loss	π over-estimated	Halve π , retry
Rare class AP doesn't improve with EFL	Scale factor too low	Raise <code>scale_factor</code> from 8 to 12
Frequent class AP drops with EFL	Scale factor too high	Reduce to 4
Overall mAP regresses with RFS	<code>t</code> too aggressive	Lower to 0.0001
Soft Teacher makes things worse	Teacher confidence threshold too low	Raise from 0.5 to 0.7
Loss goes NaN early in training	LR too high or bad data sample	Reduce LR 10x; inspect bad batches
Val mAP@0.5 high but mAP@[.50:.95] low	Localization is loose	Multi-scale training + SAHI + WBF, not NMS
Val mAP higher than LB mAP by 3+	Val/test distribution shift	Enable multi-scale training + SAHI at inference
Fold variance > 2 mAP across 5 folds	Stratification failed for rare classes	Re-stratify using rarest present class
SAHI slower without mAP gain	Tiles too small/large	Match slice size to median object side $\times$ 10
TTA degrades performance	One augmentation produces garbage (wrong rotation handling)	Remove vertical flip or rotations, keep hflip only
Ensemble not improving over best single	Using NMS instead of WBF	Switch to Weighted Box Fusion

Rules Throughout

1. **One variable per experiment.** Otherwise no attribution.
2. **Save weights after every promising run** to `/workspace/weights/` on RunPod. Kaggle resets. Pods can be deleted.
3. **Study failure cases after each phase.** Per-class AP, confusion matrix, visualized false positives/negatives. These tell you what to try next better than the aggregate mAP.
4. **Read the 2025 winning solution.** Kaggle discussion board.
5. **Stop RunPod pods when idle.** Running = charging even at rest.
6. **Submit early and often.** Every submission validates the pipeline.
7. **Track mAP@[.50:.95] always, not mAP@0.5.** If you optimize the wrong metric, you'll pick the wrong techniques.

References

- **Kiryo et al. 2017** — Positive-Unlabeled Learning with Non-Negative Risk Estimator (NeurIPS)
- **Li et al. 2022** — Equalized Focal Loss for Dense Long-Tailed Object Detection (CVPR)
- **Xu et al. 2021** — End-to-End Semi-Supervised Object Detection with Soft Teacher (ICCV)
- **Gupta et al. 2019** — LVIS: A Dataset for Large Vocabulary Instance Segmentation (RFS)
- **Akyon et al. 2022** — Slicing Aided Hyper Inference (SAHI)
- **Solovyev et al. 2021** — Weighted Boxes Fusion (WBF)
- **Caron et al. 2021** — Emerging Properties in Self-Supervised Vision Transformers (DINO)
- **FathomNet 2025 winning solution** — Kaggle discussion board
- **COCO Detection Evaluation** — cocodataset.org/#detection-eval
- **Ultralytics docs** — docs.ultralytics.com
- **BioClip2** — imageomics/bioclclip-2 on Hugging Face